SHADOW READER

Performance benchmark

Virtualized reading, scroll-position accuracy, and resource footprint

43.4%	99.8%	30.0%
lower median time-to-readable on Shadow Slave Volume 7 (n=3/build)	fewer reader-area DOM child elements on Shadow Slave Volume 7	lower median Chromium process-tree PSS on Shadow Slave Volume 7 (n=3/build)

Executive summary

This report records local benchmark results for Shadow Reader 1.0.0. The measured improvements include fewer live reading-area elements, faster time to a readable view for long text, lower measured Chromium process-tree proportional set size (PSS) in this environment, and correction of a scroll anchoring defect that previously displaced the page farther than requested and sometimes in the wrong direction.

The opening/DOM/memory comparison uses a pre-virtualized baseline versus Shadow Reader 1.0.0. The scroll comparison uses the previous virtualized build before scroll-position corrections versus the current release. These measure combined build differences; they do not isolate the effect of each individual code change.

Headline metrics

Metric	Baseline	Shadow Reader 1.0.0	Observed difference
Shadow Slave Volume 7 time-to-readable (median)	2.316 s	1.311 s	43.4% lower
Shadow Slave Volume 7 reader-area child elements	21,276	45	99.8% fewer
Shadow Slave Volume 7 Chromium process-tree PSS	497.8 MB	348.6 MB	30.0% lower
Shadow Slave Volume 7 app scroll corrections / 24 events	35 (median)	0	No app-issued corrections

Decision: no additional mandatory core-code change was identified in this review. The release is branded Shadow Reader; previous browser storage keys and the legacy port variable remain supported.

1. Opening performance and DOM footprint

Time-to-readable is measured from local text submission to a usable reading view in a fresh headless Chromium context. DOM counts are the number of child elements under the reading area after the document is ready; they indicate live DOM footprint, not total memory.

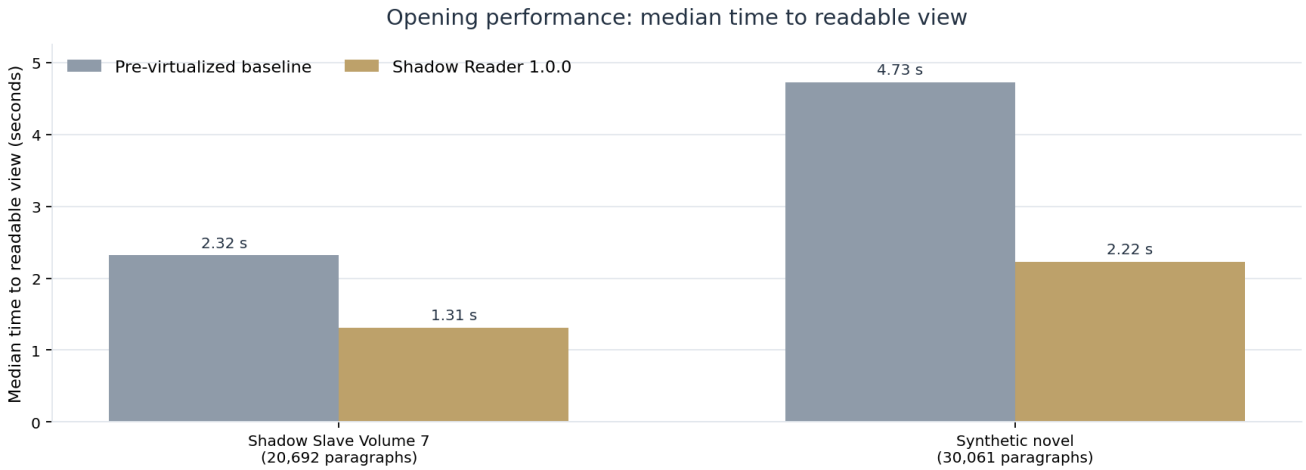


Figure 1. Median opening time. Shadow Slave Volume 7 uses three trials per build; the synthetic 30,000-paragraph workload uses two trials per build.

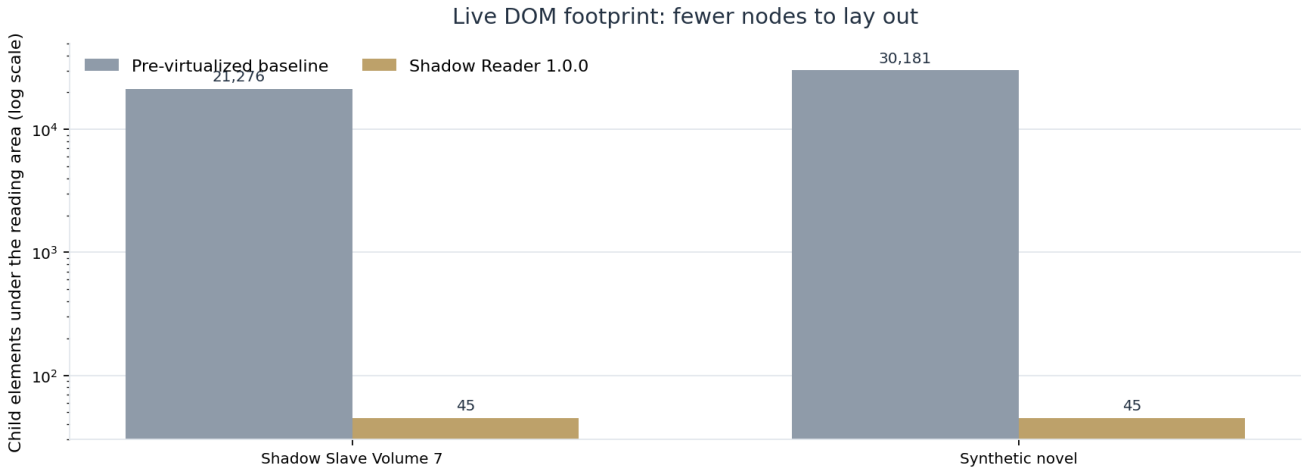


Figure 2. Live reading-area child elements. The chart uses a logarithmic y-axis so both large and small counts are visible. For Shadow Slave Volume 7, live child elements fell from 21,276 to 45. Virtualization keeps around 40 paragraph items mounted near the reading window, plus supporting elements.

2. Scroll correctness and process memory

The scripted scroll test sends 18 downward inputs of +250 px, then 6 reverse inputs of -200 px. The expected first-phase movement is 4,500 px down, followed by a requested 1,200 px in the reverse direction. This isolates scroll-position correction behavior; it is not a physical touchpad inertia measurement.

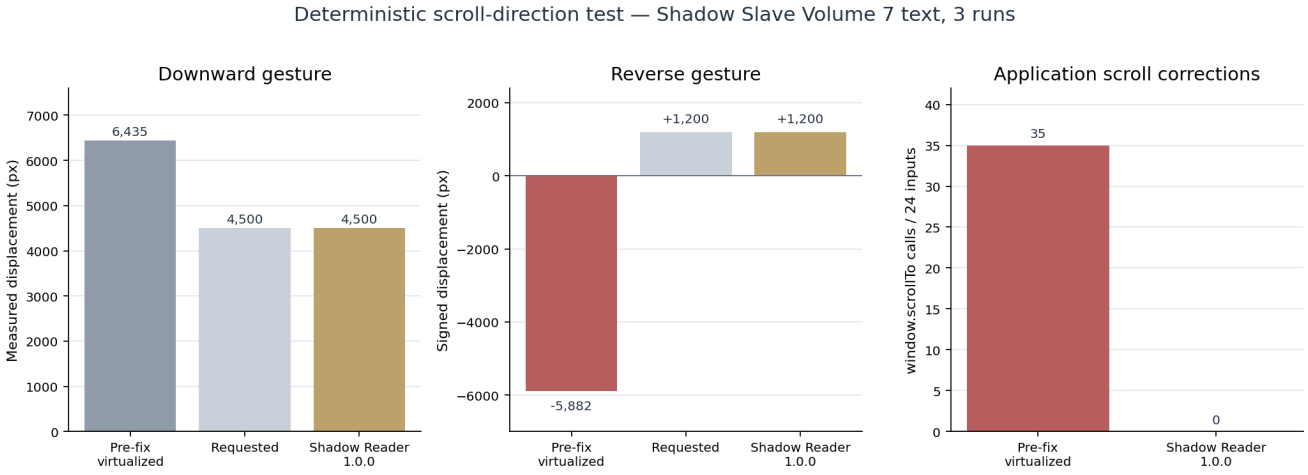


Figure 3. The pre-fix virtualized build overshot the downward movement and continued in the wrong direction during reverse input. Shadow Reader matched the injected displacement and issued zero application `window.scrollTo()` corrections in three Shadow Slave Volume 7 trials.

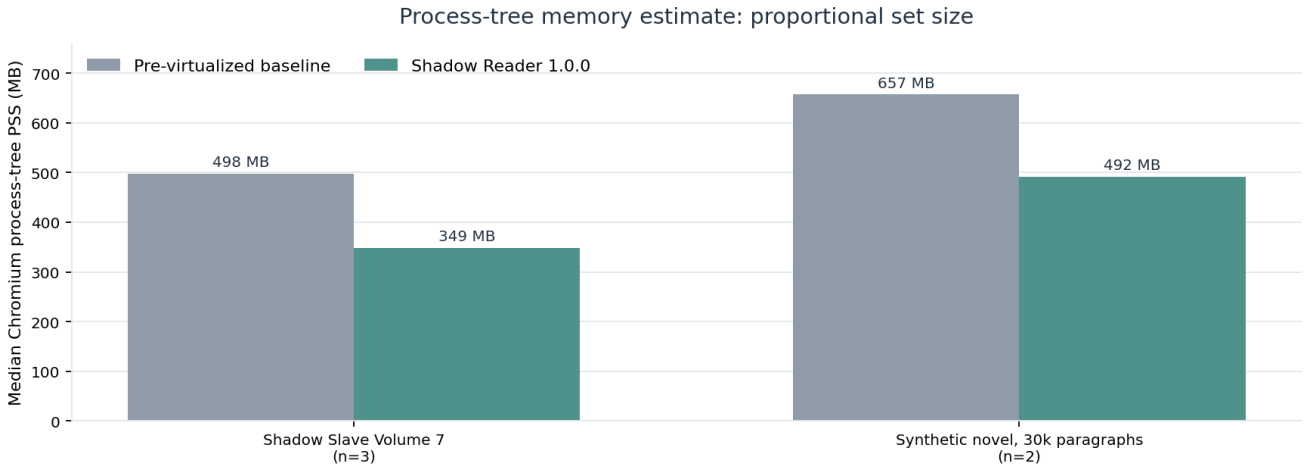


Figure 4. Median process-tree proportional set size (PSS) across Chromium and its child processes. Shadow Slave Volume 7 uses n=3 runs per build; the synthetic 30,000-paragraph workload uses n=2. PSS apportions shared pages and is preferred here over summing RSS, which can double-count shared memory.

Memory caveat: JavaScript heap measurements alone were higher and variable in the virtualized candidate. This reduction is a Linux Chromium process-tree PSS result, not a direct measurement of Firefox or total system memory on the target laptop.

3. Test environment, strain tests, and limitations

Environment

Component	Recorded environment
Operating system	Debian GNU/Linux 13 (Trixie), containerized x86_64; kernel 6.18.55
CPU / RAM	AMD EPYC 9V74 virtual CPU; 5 vCPUs; 5.8 GiB visible RAM; 0 B swap
Runtime / browser	Python 3.13.5; Node.js v22.16.0; Chromium 144.0.7559.96; Python Playwright 1.57.0
PDF text extraction	Poppler pdftotext 25.06.0
Viewport	1366 x 768 CSS pixels; headless Chromium
Source workload	Text extracted locally from Shadow Slave Volume 7; synthetic 8,000- and 30,000-paragraph text; Alice and Shadow Slave Volume 6 used for feature regressions

Strain tests

Scenario	Result	Interpretation
8,000 paragraphs, 4x CPU throttle	Pass	6,000 px down / 1,300 px up matched; zero app corrections; about 40 paragraph items mounted.
30,000 paragraphs, 2x CPU throttle	Pass (one run)	Readable in 4.89 s; directional scroll displacement matched; zero app corrections. Single run; not used for median claim.
30,000 paragraphs, 4x CPU throttle	Inconclusive	Automation did not finish within the run budget. Not counted as a pass.

Regression suite on the release source

Passed: 296 parser groups when the three full-text test inputs were supplied (Alice 12 chapters, Shadow Slave Volume 6 170 chapters, Shadow Slave Volume 7 354 chapters); 27 Continue Reading/progress checks; 10 Focus Here checks; 10 branding/compatibility checks; 12 server API checks; virtualization browser smoke; deep-scroll performance smoke; touchpad-like scroll regression; Focus Here browser smoke; Continue Reading browser smoke; Python compilation and shell syntax checks. The test PDFs themselves are not included with the release.

Limitations and responsible claims

All performance charts are from a containerized Linux environment using headless Chromium, not the target Intel i3 / 8 GB laptop running native Firefox. The injected wheel test is deterministic and does not measure physical touchpad inertia or perceived smoothness. Trial counts are small (n=2 or n=3); these are engineering measurements, not statistical significance tests. The 30,000-paragraph / 4x CPU case is inconclusive. No frame-rate uplift is claimed. For promotional use, qualify the figures as results from this specific benchmark environment and avoid universal claims.

Reproduction materials are included under `benchmarks/`: raw trial CSV, environment/source metadata, charts, method notes, and a reusable Playwright benchmark script. Source hashes in `environment.json` identify the branded HTML/server revision used for the package.